

Módulo 8: Notificaciones, Documentos y Soporte

Grupo 07 Estado: Arquitectura Base Configurada

Repositorio:
Adonai1027/TPI_DS_07_2026
Fecha: Septiembre 2026
Stack: .NET 8 / C# / Docker /
Postgres

1. Alcance y Responsabilidades de Nuestro Módulo (M8)

En la plataforma de movilidad urbana distribuida, el **Grupo 07** es responsable exclusivo del **Módulo 8**. Es un microservicio orientado a eventos que asiste al ciclo de vida del viaje y a los demás módulos:

Requerimiento	Nombre	Descripción Funcional
RF-8.1 / 8.2	Notificaciones	Envío de avisos automáticos ante hitos de viaje y reservas (solicitud, arribo, inicio, cobro) por Email, Push o SMS.
RF-8.3	QR de Verificación	Generación de código QR seguro, temporal y de un solo uso para autorizar el inicio del viaje entre pasajero y chofer.
RF-8.4 / 8.5	Comprobante PDF	Generación dinámica de comprobantes de pago/viaje en formato PDF descargable y servicio de reenvío por correo.
RF-8.6	Trazabilidad y Reintentos	Monitoreo de estado de entrega de notificaciones (Pendiente, Enviado, Fallido) y política de reintentos controlados.
RF-8.7	Tickets de Soporte	Sistema CRUD para gestionar reclamos de usuarios asociados a un viaje, reserva o pago específico.
RF-8.8	Consumo Asíncrono	Procesamiento desacoplado de eventos recibidos por cola de mensajería (RabbitMQ) sin bloquear el flujo principal.

2. Decisiones de Arquitectura y Stack Tecnológico

Aprovechando la **Libertad Tecnológica (RNF-01)**, se seleccionó un ecosistema enterprise moderno, de alta demanda laboral y robusto para coordinar a 9 personas sin fricciones:

Lenguaje y Framework:	C# con .NET 8 (Web API). Tipado estático estricto para evitar errores en tiempo de ejecución, inyección de dependencias nativa y alto rendimiento.
Contratos de API (RNF-03):	OpenAPI Specification (Swagger). Documentación viva e interactiva autogenerada a partir del código.
Base de Datos (RNF-05):	PostgreSQL 16 gestionado mediante Entity Framework Core 8 (EF Core) con migraciones Code-First automáticas.
Mensajería (RNF-10):	RabbitMQ 3.13 para la recepción de eventos asíncronos de Viajes (M6), Despacho (M5) y Pagos (M7).

Caché y TTL (RNF-11):

Redis 7 para almacenamiento en memoria de tokens QR con tiempo de vida configurable (expiración automática de 10 min).

Contenedores (RNF-04):

Docker & Docker Compose con builds multi-etapa para levantar todo el entorno con un solo comando.

3. Organización del Equipo: 3 Células de Trabajo (3 Personas c/u)

Para evitar que 9 personas editen los mismos archivos y sufran colisiones en Git, el código se organizó bajo la arquitectura **Vertical Slices (Carpetas por Funcionalidad)**. Cada célula es dueña de su dominio:

Célula 1: Notificaciones

Requerimientos: RF-8.1, 8.2, 8.6, 8.8

- Consumidores RabbitMQ (MassTransit)
- Servicio de envío de Emails con plantillas
- Notificaciones Push y SMS
- Lógica de reintentos y auditoría

Carpeta:

Features/Notificaciones

Célula 2: Documentos & QR

Requerimientos: RF-8.3, 8.4, 8.5, RNF-11

- Diseño de factura/recibo en PDF con QuestPDF
- Generador de códigos QR con QRCoder
- Tokens temporales en Redis con TTL
- Descarga y reenvío de comprobantes

Carpeta: Features/Documentos

Célula 3: Soporte & Datos

Requerimientos: RF-8.7, RNF-03, RNF-05

- CRUD de Tickets de Soporte
- Transiciones de estado de reclamos
- Modelo relacional y migraciones EF Core
- Mantenimiento de contratos OpenAPI

Carpeta: Features/Soporte

4. Metodología de Trabajo Profesional (Scrumban + Git Flow)

- **Sprints de 2 semanas:** Cada sprint concluye con una demo funcional de los requerimientos entregables.
- **Tablero Kanban (GitHub Projects):** Columnas: *Backlog* → *To Do* → *In Progress* → *In Review (PR)* → *Done*.
- **Regla de oro de Git:** La rama `main` está protegida. **Nunca se comitea directo a main.**
- **Flujo de trabajo diario:**
 1. Actualizar `main`: `git pull origin main`
 2. Crear rama de tarea: `git checkout -b feature/nombre-tarea`
 3. Hacer commits descriptivos (Conventional Commits).
 4. Subir la rama y abrir un **Pull Request** en GitHub con al menos 1 revisión aprobada antes del merge.

5. Guía de Onboarding Paso a Paso (Para los 8 Compañeros)

Cualquier integrante nuevo del grupo puede dejar su entorno listo en 10 minutos siguiendo estos pasos:

1. **Aceptar invitación en GitHub:** Revisar el correo y confirmar el acceso al repositorio `Adonai1027/TPI_DS_07_2026`.
2. **Instalar herramientas esenciales:**
 - **.NET 8 SDK:** Desde dotnet.microsoft.com o con `winget install Microsoft.DotNet.SDK.8`.
 - **Docker Desktop:** Con soporte WSL2 activado.
3. **Clonar el repositorio:**

```
git clone https://github.com/Adonai1027/TPI_DS_07_2026.git
cd TPI_DS_07_2026
```

4. Levantar la infraestructura (Base de Datos, RabbitMQ y Redis):

```
docker compose up -d
```

(Panel de RabbitMQ disponible en: <http://localhost:15672> • Usuario: `guest` / Pass: `guest`)

5. Ejecutar la API y verificar Swagger:

```
dotnet run --project src/M8.Notificaciones.Api
```

Abre el navegador en la URL indicada en consola (ej. `http://localhost:5247`) y verás Swagger con todos los endpoints listos para probar. La base de datos PostgreSQL se sincronizará automáticamente al iniciar.

✅ **Estado al día de hoy:** Arquitectura inicial creada, solución .NET 8 configurada con OpenAPI, Docker Compose operativo con PostgreSQL/RabbitMQ/Redis, persistencia EF Core conectada con migraciones automáticas y todo el código sincronizado en la rama `main` de GitHub.